

《数据库系统原理》报告

数据库系统原理作业 2

张鲈泮

北京交通大学计算机科学与技术学院 AI2201 班 22281052@bjtu.edu.cn

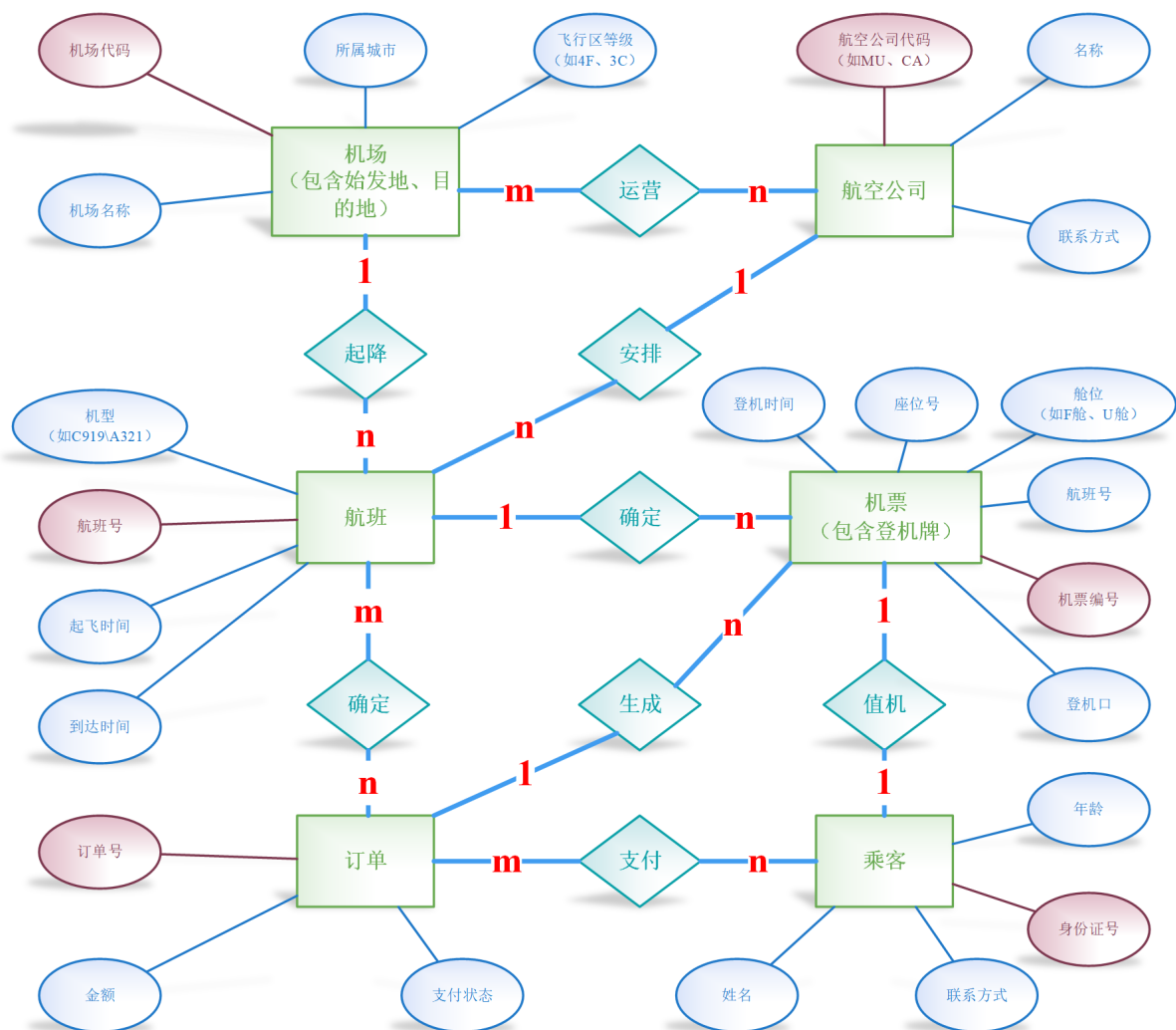


图 1: 航空公司航班查询业务模型 E-R 图, 来自于作业 1, 并进行了一定修正改进

1 题目 1

题目要求: 针对作业 1 的航空公司航班查询业务模型, 尝试标识该业务场景中的数据模式, 即列出该业务涉及到的所有实体型 (包括实体名及属性, 可以适当简化)。采用如下形式列出:

实体型名 I (属性 $1, \dots, \text{属性 } n$)

尝试在你选定的数据库平台 (MySQL) 中建立实体型所对应的表, 并想办法往其中插入数据, 验证你所设计的模型的合理性。

根据作业 1 与 ER 图 (图1), 可以以下是该业务场景涉及的主要实体类型及其属性:

• 机场 (Airport)

1. 机场代码 (airport_code, 主键)
2. 机场名称 (airport_name)
3. 所属城市 (city)
4. 飞行区等级 (flight_zone)

• 航空公司 (Airline)

1. 公司代码 (airline_code, 主键)
2. 名称 (airline_name)
3. 联系方式 (contact_info)

• 航班 (Flight)

1. 航班号 (flight_number, 主键)
2. 机型 (aircraft_model)
3. 起飞时间 (departure_time)
4. 到达时间 (arrival_time)
5. 出发机场 (departure_airport, 外键关联 Airport)
6. 目的机场 (arrival_airport, 外键关联 Airport)
7. 航空公司 (airline_code, 外键关联 Airline)

• 机票 (Ticket)

1. 机票编号 (ticket_id, 主键)
2. 航班号 (flight_number, 外键关联 Flight)
3. 身份证号 (id_card, 外键关联 Passenger)
4. 座位号 (seat_number)
5. 登机口 (gate)
6. 登机时间 (boarding_time)
7. 舱位 (cabin_seat)

• 订单 (Orderinfo)

1. 订单号 (order_id, 主键)
2. 身份证号 (id_card, 外键关联 Passenger)
3. 金额 (total_amount)
4. 支付状态 (payment_status)

- 乘客 (Passenger)

1. 身份证号 (id_card, 主键)
2. 姓名 (name)
3. 年龄 (age)
4. 联系方式 (contact_info)

之后, 我用以下语句创建了 Database, 并且创建了各实体对应的表。

```
1 # 创建数据库
2 CREATE DATABASE AirlineDB;
3 USE AirlineDB;
4
5 # 机场表
6 CREATE TABLE Airport (
7     airport_code VARCHAR(10) PRIMARY KEY,
8     airport_name VARCHAR(100) NOT NULL,
9     city VARCHAR(50) NOT NULL,
10    flight_zone VARCHAR(5) NOT NULL
11 );
12
13 # 航空公司表
14 CREATE TABLE Airline (
15     airline_code VARCHAR(10) PRIMARY KEY,
16     airline_name VARCHAR(100) NOT NULL,
17     contact_info VARCHAR(100)
18 );
19
20 # 航班表
21 CREATE TABLE Flight (
22     flight_number VARCHAR(10) PRIMARY KEY,
23     aircraft_model VARCHAR(20) NOT NULL,
24     departure_time DATETIME NOT NULL,
25     arrival_time DATETIME NOT NULL,
26     departure_airport VARCHAR(10),
27     arrival_airport VARCHAR(10),
```

```

28     airline_code VARCHAR(10),
29     FOREIGN KEY (departure_airport) REFERENCES Airport(airport_code),
30     FOREIGN KEY (arrival_airport) REFERENCES Airport(airport_code),
31     FOREIGN KEY (airline_code) REFERENCES Airline(airline_code)
32 );
33
34 # 乘客表
35 CREATE TABLE Passenger (
36     id_card VARCHAR(20) PRIMARY KEY,
37     name VARCHAR(50) NOT NULL,
38     age INT CHECK (age >= 0),
39     contact_info VARCHAR(100)
40 );
41
42 # 机票表
43 CREATE TABLE Ticket (
44     ticket_id INT AUTO_INCREMENT PRIMARY KEY,
45     flight_number VARCHAR(10),
46     id_card VARCHAR(20),
47     seat_number VARCHAR(10),
48     gate VARCHAR(10),
49     boarding_time DATETIME,
50     cabin_seat VARCHAR(10),
51     FOREIGN KEY (flight_number) REFERENCES Flight(flight_number),
52     FOREIGN KEY (id_card) REFERENCES Passenger(id_card)
53 );
54
55 # 订单表
56 CREATE TABLE Orderinfo (
57     order_id INT AUTO_INCREMENT PRIMARY KEY,
58     id_card VARCHAR(20),
59     total_amount DECIMAL(10,2),
60     payment_status ENUM('Pending', 'Completed', 'Failed'),
61     FOREIGN KEY (id_card) REFERENCES Passenger(id_card)
62 );

```

```
mysql> show tables;
+-----+
| Tables_in_airlinedb |
+-----+
| airline              |
| airport              |
| flight               |
| orderinfo            |
| passenger             |
| ticket                |
+-----+
6 rows in set (0.00 sec)
```

(a) 查询 DB 中所有表

```
mysql> desc Airline
-> ;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| airline_code   | varchar(10)   | NO   | PRI | NULL    |       |
| airline_name   | varchar(100)  | NO   |     | NULL    |       |
| contact_to     | varchar(100)  | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

(b) 查询 Airline 表

图 2: 创建 AirlineDB 数据库、表并进行配置

图2展示了创建 AirlineDB 数据库与相关表的情况。之后，在插入了一些模拟数据后，我验证了设计的数据模型是否合理。使用的语句如下：

1. **验证实体约束（主键唯一性）**，就是确保表中主键字段不可重复且不可为空。我运用了下面的代码进行验证。由图3知，返回结果为空，说明主键约束有效，未发生重复。

```
1  # 检查机场主键唯一性
2  SELECT airport_code, COUNT(*) FROM Airport GROUP BY
   airport_code HAVING COUNT(*) > 1;
3
4  # 检查乘客身份证号唯一性
5  SELECT id_card, COUNT(*) FROM Passenger GROUP BY id_card
   HAVING COUNT(*) > 1;
6
7  # 检查航班号唯一性
8  SELECT flight_number, COUNT(*) FROM Flight GROUP BY
   flight_number HAVING COUNT(*) > 1;
```

```
mysql> SELECT airport_code, COUNT(*) FROM Airport GROUP BY airport_code HAVING COUNT(*) > 1;
Empty set (0.01 sec)

mysql> SELECT id_card, COUNT(*) FROM Passenger GROUP BY id_card HAVING COUNT(*) > 1;
Empty set (0.00 sec)

mysql> SELECT flight_number, COUNT(*) FROM Flight GROUP BY flight_number HAVING COUNT(*) > 1;
Empty set (0.00 sec)
```

图 3: 验证实体约束（主键唯一性）

2. **验证外键约束**，也就是说确保外键值必须来自被引用表的主键集合。我运用了下面的代码进行验证。由图4知，返回结果为空，说明外键约束有效，未发生没有关联的情况。

```
1  # 检查是否存在航班表中未关联机场的记录
2  SELECT * FROM Flight
```

```

3      WHERE departure_airport NOT IN (SELECT airport_code FROM
      Airport)
4      OR arrival_airport NOT IN (SELECT airport_code FROM
      Airport);
5
6      # 检查是否存在航班表中未关联航空公司的记录
7      SELECT * FROM Flight
8      WHERE airline_code NOT IN (SELECT airline_code FROM Airline)
9      ;
10
11     # 检查是否存在机票中未关联航班或乘客的记录
12     SELECT * FROM Ticket
13     WHERE flight_number NOT IN (SELECT flight_number FROM Flight
14     )
15     OR id_card NOT IN (SELECT id_card FROM Passenger);
16
17     # 检查订单中是否存在未关联乘客的记录
18     SELECT * FROM Orderinfo
19     WHERE id_card NOT IN (SELECT id_card FROM Passenger);

```

```

mysql> SELECT * FROM Flight
      -> WHERE departure_airport NOT IN (SELECT airport_code FROM Airport)
      -> OR arrival_airport NOT IN (SELECT airport_code FROM Airport);
Empty set (0.00 sec)

mysql> SELECT * FROM Flight
      -> WHERE airline_code NOT IN (SELECT airline_code FROM Airline);
Empty set (0.00 sec)

mysql> SELECT * FROM Ticket
      -> WHERE flight_number NOT IN (SELECT flight_number FROM Flight)
      -> OR id_card NOT IN (SELECT id_card FROM Passenger);
Empty set (0.00 sec)

mysql> SELECT * FROM Orderinfo
      -> WHERE id_card NOT IN (SELECT id_card FROM Passenger);
Empty set (0.00 sec)

```

图 4: 验证外键约束

3. **验证逻辑合理性**，例如验证时间关系，确保航班到达时间晚于起飞时间、登机时间早于起飞时间等逻辑。由图5知，返回结果为空，说明外键约束有效，未发生没有关联的情况。

```

1      # 航班起飞时间应早于到达时间
2      SELECT flight_number, departure_time, arrival_time
3      FROM Flight
4      WHERE departure_time >= arrival_time;

```

```

5
6      # 机票登机时间应早于该航班起飞时间
7      SELECT T.ticket_id, T.boarding_time, F.departure_time
8      FROM Ticket T
9      JOIN Flight F ON T.flight_number = F.flight_number
10     WHERE T.boarding_time >= F.departure_time;

```

```

mysql> SELECT flight_number, departure_time, arrival_time
-> FROM Flight
-> WHERE departure_time >= arrival_time;
Empty set (0.00 sec)

mysql> SELECT T.ticket_id, T.boarding_time, F.departure_time
-> FROM Ticket T
-> JOIN Flight F ON T.flight_number = F.flight_number
-> WHERE T.boarding_time >= F.departure_time;
Empty set (0.00 sec)

```

图 5: 验证逻辑合理性

4. **进行模拟完整查询：**乘客与航班、订单信息联查。由图6知，返回结果展示了我加入的模拟数据，能够成功进行信息联查。

```

1      SELECT
2          P.name,
3          P.id_card,
4          F.flight_number,
5          F.departure_time,
6          F.arrival_time,
7          T.seat_number,
8          T.cabin_seat,
9          O.order_id,
10         O.total_amount,
11         O.payment_status
12     FROM Passenger P
13     JOIN Ticket T ON P.id_card = T.id_card
14     JOIN Flight F ON T.flight_number = F.flight_number
15     JOIN Orderinfo O ON P.id_card = O.id_card;

```

```
mysql> SELECT
-> P.name,
-> P.id_card,
-> F.flight_number,
-> F.departure_time,
-> F.arrival_time,
-> T.seat_number,
-> T.cabin_seat,
-> O.order_id,
-> O.total_amount,
-> O.payment_status
-> FROM Passenger P
-> JOIN Ticket T ON P.id_card = T.id_card
-> JOIN Flight F ON T.flight_number = F.flight_number
-> JOIN Orderinfo O ON P.id_card = O.id_card;
```

name	id_card	flight_number	departure_time	arrival_time	seat_number	cabin_seat	order_id	total_amount	payment_status
李四	110102199202021234	CA1502	2025-06-11 09:00:00	2025-06-11 11:15:00	18C	C	2	1200.00	Pending
张三	110101199001011234	MU5101	2025-06-10 08:00:00	2025-06-10 10:30:00	12A	Y	1	800.00	Completed

2 rows in set (0.01 sec)

图 6: 进行模拟完整查询

通过以上内容，能够较为完整地验证数据模型的设计是逻辑正确的，完整性约束是生效的。

2 题目 2

题目要求：请自学了解 *Axure RP* 等快速原型设计工具的概念及功能，并尝试下载一个试用版。请你充当产品经理和产品界面模型设计工程师，模仿或创新设计两个用户界面（*UI*、用户视图或外模式的子模式）。请提交原始设计文档以证明你做了此项工作。

我下载了 *Axure RP 9* 软件，进行了安装。我以“航空公司航班查询系统”的业务为基础，设计了一个移动端 APP 原型，涵盖用户从登录、查询航班、查看行程到管理个人信息的全流程操作。



图 7: UI 主要界面

启动页点击“跳过”，或者等待 3 秒后后进入登录界面。用户通过手机号或邮箱 + 密码组合进行身份验证，并可跳转至注册或找回密码模块（图 8a-8d）。在登录成功后进

入首页（图7a），顶部是主要导航模块，包括行程、航班查询、机场大屏、航空公司等功能入口，中间展示近期航班行程，底部是基于目的地的图文推荐，增强用户粘性。

在“行程”模块中（图7b），用户可以查看自己未来或历史出行记录。每一段行程以卡片形式展示航班号、起降时间、机场及航空公司等信息，点击后可进一步查看订单或机票详情。这部分数据主要对应数据库中的 Flight、Order、Ticket 等表，信息通过用户 id_card 字段进行关联查询。

“航班动态”模块（图7c）支持多方式查询，用户可以根据航班号、起降地或机型编号等条件检索具体航班，并获取准点率、延误率等统计指标，并且设计了“机场大屏”功能，可以查询机场整体运行状态。这部分前端展示的数据来自于 Flight、Airport、Airline 等实体，在后端通过字段匹配查询后转化为用户可视化界面。

在“我的”页面中（图8e），用户可查看和修改个人信息，包括用户名、邮箱、联系方式、性别和出生日期等字段（图7d），数据与 Passenger 表中的字段一一对应。还提供通知管理、语言切换、帮助中心等常见辅助功能，并支持退出登录等基础操作。



图 8: UI 次要界面

3 题目 3

题目要求：请通过你所设计的用户界面所对应的外模式子集，以及该子集与各种模式间的映射关系，思考并叙述数据库系统从内模式、模式、外模式并最终到用户界面的映射 (*mapping*) 过程，分析过程中可能会涉及到的数据加工环节。分别指出一般需要由程序员写程序实现的环节和由数据库管理系统实现的环节。

在作业航空公司航班查询业务数据库系统设计中，采用的是数据库系统三层架构，即**内模式**、**模式**、**外模式**，并最终通过用户界面 (UI) 与用户进行交互。本次设计中，具体映射过程为：

- **内模式（物理存储）：**内模式是数据库物理实现的最底层结构，主要涉及数据文件如何存储、索引建立、数据分区、磁盘管理等。该环节主要由 **DBMS** 自动实现，程序员一般不需干涉。

- **模式（概念模式）**：模式是对数据库结构的整体描述，是数据的逻辑组织形式。此次作业中设计的 Airport、Airline、Flight、Passenger、Order、Ticket 等实体型及其属性、主键和外键关系，即构成了数据库的模式结构。在实现阶段通过 SQL 语句，如 CREATE TABLE 建立数据表，并通过外键约束确保数据参照完整性。这些工作需要**程序员主动编写**并实现。
- **外模式（用户视图）**：外模式是面向不同用户的个性化数据子集，反映用户所需的特定视图。以问题 2 移动端 APP 界面为例。首页展示的“近期日程”来自于用户关联的航班和订单数据，提供的是 Order 表、Flight 表、Passenger 表联合查询的视图；航班动态查询页面的数据，则是 Flight 表及机场、航空公司相关数据的联合视图；行程管理页面展示的是基于用户 id_card 关联的订单和航班信息的视图；个人中心页面则提供 Passenger 表的数据视图，并允许编辑和更新。

上述外模式的设计与维护，需要**程序员和前端工程师编写**对应的 SQL 语句、后端逻辑代码，以及前端界面逻辑代码，实现从数据库获取数据并渲染至界面，完成外模式到模式的映射过程。

总结而言，**SQL 编写、数据组合与加工逻辑由程序员实现，数据库管理系统 (DBMS) 负责底层数据的高效存储和检索操作。**

在具体实现过程中，从数据库的数据到最终呈现给用户界面，可能涉及以下数据加工环节：

1. **数据查询组合**：用户界面请求后端服务接口，后端编写 SQL 从数据库抽取数据，如订单详情页面的联表查询。
2. **数据过滤筛选**：比如根据用户身份、订单状态、时间区间等条件过滤数据。
3. **数据格式转换**：例如航班准点率统计指标的计算，可能通过聚合函数（COUNT、AVG 等）生成可视化指标。
4. **接口封装 JSON 化**：后端将数据库查询结果封装成 JSON 格式接口给前端，前端解析后渲染页面。

4 题目 4

题目要求：请尝试学习掌握数据库系统数据规模（条或记录数）估算方法，估计所选业务场景的数据规模，针对这种规模，请说明这种规模的数据可能对内模式的实现或表示存在什么要求。

根据航空公司航班查询业务涉及的实体类型及其属性，具体估算各实体的数据记录条数，并进行数据存储空间的初步估计。假设本系统服务全国约 100 个机场，每日平均 300 个航班，航班平均乘客量为 150 人，平均每位乘客每年生成 2 个订单。

首先计算数据规模（每年新增）：

- **机场 (Airport)**：100 条（全国 100 个机场）；

- **航空公司 (Airline)**: 约 50 条 (全国主要航空公司数量);
- **航班 (Flight)**: 100 个机场 $\times$ 每天 300 航班 $\times$ 365 天 $\approx$ 1100 万条;
- **乘客 (Passenger)**: 全年航班总数 $\times$ 每航班乘客数 (去重后), 约为 16.5 亿 $\times$ 10% 去重估算 $\approx$ 1.65 亿条;
- **机票 (Ticket)**: 航班数 $\times$ 乘客数 = 1100 万 $\times$ 150 $\approx$ 16.5 亿条;
- **订单 (Orderinfo)**: 乘客数 $\times$ 每年平均订单数 = 1.65 亿 $\times$ 2 $\approx$ 3.3 亿条。

实体数据规模条数估算结果如下:

表 1: 主要实体类型的数据规模估算表 (每年新增)

实体名称	每条记录平均大小 (Bytes)	预计记录条数 (条)
机场 (Airport)	100	100
航空公司 (Airline)	150	50
航班 (Flight)	200	11,000,000
乘客 (Passenger)	150	165,000,000
机票 (Ticket)	200	1,650,000,000
订单 (Orderinfo)	100	330,000,000

进一步估算数据存储空间需求 (不含索引), 计算公式如下:

$$\text{总空间大小 (Bytes)} = \text{单条记录平均大小 (Bytes)} \times \text{记录总数} \quad (1)$$

各实体空间计算:

- 机场: $100 \text{ Bytes} \times 100 = 10 \text{ KB}$
- 航空公司: $150 \text{ Bytes} \times 50 = 7.5 \text{ KB}$
- 航班: $200 \text{ Bytes} \times 11,000,000 \approx 2.2 \text{ GB}$
- 乘客: $150 \text{ Bytes} \times 165,000,000 \approx 24.75 \text{ GB}$
- 机票: $200 \text{ Bytes} \times 1,650,000,000 \approx 307.5 \text{ GB}$
- 订单: $100 \text{ Bytes} \times 330,000,000 \approx 30.75 \text{ GB}$

汇总后, 本业务每年新增数据的存储空间约为:

$$2.2 \text{ GB} + 24.75 \text{ GB} + 307.5 \text{ GB} + 30.75 \text{ GB} \approx 365.2 \text{ GB} \quad (2)$$

经上述计算，该航空公司航班查询业务每年数据增长规模约在 **16.5 亿条记录以上**，存储需求（不含索引、日志、备份等额外空间）约为 **365GB 级别**。这种规模的数据对数据库内模式的存储优化、索引策略、缓存机制以及数据库分区等技术均提出较高要求。面对这种规模的数据量，数据库的内模式实现将面临巨大挑战，必须采用特殊的技术手段。因此针对上述数据规模，数据库内模式的实现可以考虑以下几种技术手段：

首先，针对单表数据量庞大的问题，应采用**数据分区（Partitioning）策略**。例如，按照日期、机场代码、航班号等关键字段，将航班、机票、订单表**划分成多个子表或分片**，显著降低单表的查询负载并提高系统响应速度。为保障大数据环境下高效的查询性能，数据库应采用合理的**索引（Index）策略**。如主键字段（航班号、身份证号）自动建立聚集索引，其他高频查询字段（如航班日期、机场代码、航空公司代码）需建立适当的二级索引。数据库内模式应实现高效的**缓存（Caching）机制**，例如引入 Redis 或 Memcached 缓存数据库，将高频查询的数据暂存于缓存中，减少数据库 I/O 负担，提升用户访问效率。

面对如此庞大的数据量，数据库单机部署无法满足性能需求，应采用**分布式数据库集群（Cluster）部署方式**，如 MySQL Cluster、OceanBase 或 TiDB 等，实现负载均衡和高可用性。同时必须建立完善的数据备份（Backup）与容灾机制，定期进行数据备份，防止数据丢失；同时对于历史数据采用定期归档策略，确保系统长期稳定运行。大规模数据存储对磁盘空间需求较大，因此内模式实现时需要考虑采用**数据压缩技术**，有效降低数据存储占用空间，并提高磁盘读写效率。

以上技术手段与策略中，**数据库分区设计、索引优化、缓存系统搭建与维护、数据备份与恢复策略**由程序员设计实现；而**数据的物理存储、索引和分区的物理维护、数据压缩、磁盘管理**则主要由数据库管理系统（DBMS）自动完成。

5 总结感想

通过本次作业，我回顾了学习了数据库系统的三层结构（内模式、模式、外模式）以及数据库设计与实现的全流程，复习了实体与属性定义、完整性约束分析多个知识点。

在原型设计部分，我通过自学快速原型工具（如 Axure RP）完成了航空公司航班查询系统的 APP 界面设计，设计涵盖启动页、登录页、航班查询、行程管理、个人信息管理等模块，通过界面设计与数据库实体的映射分析，我深刻理解了数据库三层结构的内在逻辑及数据在不同层次之间的流动与转化。

最后我通过假设与公式定量估算了航空业务的数据规模，分析了上亿条数据记录所带来的数据库存储压力与性能要求，例如数据分区、索引优化、缓存机制、分布式集群部署等，进一步增强了我对数据库系统实际应用中的性能优化与技术架构的认识。

总体而言，这次作业让我对数据库的理论原理、实际操作、系统设计方法和应用场景都有了更加全面而深入的理解，不仅巩固了数据库理论基础，也提高了实践能力，为后续进一步深入学习数据库技术和进行更复杂的数据库应用开发奠定了良好的基础。